APARTMENT MANAGEMENT SYSTEM

BULACAN STATE UNIVERSITY
BUSTOS CAMPUS

Submission Date:
October 26, 2026

BSIT 2C-G1

Subject:
Object-Oriented Programming

Submitted to:
Prof. Ragos, Ryan

Submitted by: Group 3
Bautista, Synnead Mylz R.
Santos, Marvin G.
Vicente, John Lloyd B.

TABLE OF CONTENTS

1. PROJECT OVERVIEW.....2
1.1 System Description..... 2
1.2 Project Objectives..... 2

2. SYSTEM FEATURES AND FUNCTIONALITIES.....2
2.1 Login & Access Control..... 2
2.2 Tenant Home Dashboard..... 4
2.3 Rent and Payment.....4
2.4 Account Management (User Profile)..... 5
2.5 Administrator Dashboard.....6
2.6 Interactive Pop-Up and Dialog Windows.....7

3. OBJECT-ORIENTED PROGRAMMING IMPLEMENTATION.....10
3.1 Classes and Objects..... 10
3.2 Encapsulation..... 10
3.3 Inheritance..... 11
3.4 Polymorphism and Event Handling.....11

4. DATABASE AND DATA MANAGEMENT..... 11
4.1 Centralized Data Store (DataStore.java)..... 11
4.2 User and Account Management (UserData.java and UserDataBP.java)..... 12
4.3 Data Synchronization and Updates..... 12

5.CONCLUSION..... 12

1. PROJECT OVERVIEW

The Apartment Management System is developed to make apartment operations more organized and manageable. It provides a convenient way to handle tenant information, apartment units, lease details, and rent payment transactions. The system aims to improve the accuracy and efficiency of apartment management by using a computerized process for storing information and processing transactions.

1.1 System Description

The Apartment Management System is a standalone application developed using Java Swing to help an administrator manage apartment units, tenant information, lease details, and rent payments in one organized application. Instead of allowing users to register themselves, the system uses an administrator-managed account process. The administrator creates a tenant account and assigns the tenant to an available apartment unit. The tenant can then log in using the email address and password provided during account creation.

The administrator can view a dashboard summary, manage tenants and apartment units, assign move-in and lease information, and review transaction history. Tenants can view their dashboard, personal and apartment information, edit selected profile fields, change their password, and make rent payments. The system also applies a payment rule in which overdue rent must be settled before the tenant can pay the current month’s rent.

1.2 Project Objectives

1. To organize and manage tenant profiles, lease details, and contact information.
2. To process rent calculations, payment overviews, and transaction tracking.
3. To monitor apartment unit availability and lease expirations.
4. To implement secure authentication with role-based routing for Administrators and Tenants.
5. To incorporate security safeguards including account lockout mechanisms and password-protected administrative actions.

2. SYSTEM FEATURES AND FUNCTIONALITIES

2.1 Login & Access Control

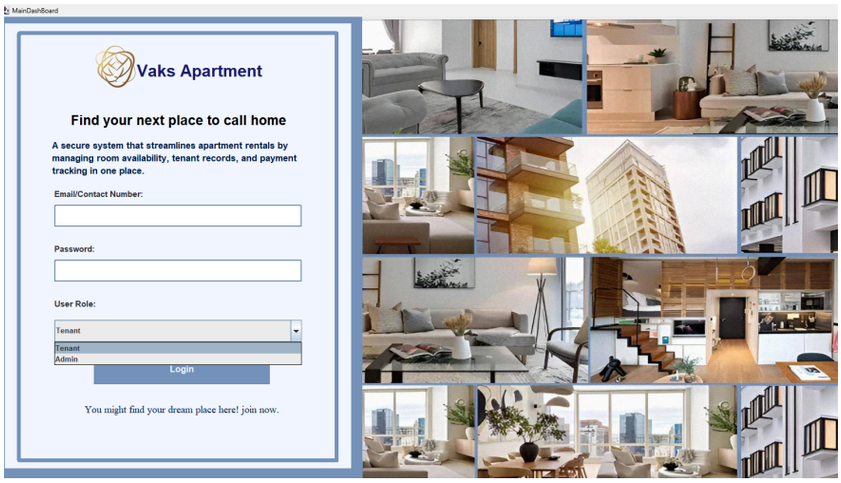


Figure 1

The primary gateway where users select their role **(Tenant or Admin**), input credentials, and authenticate.

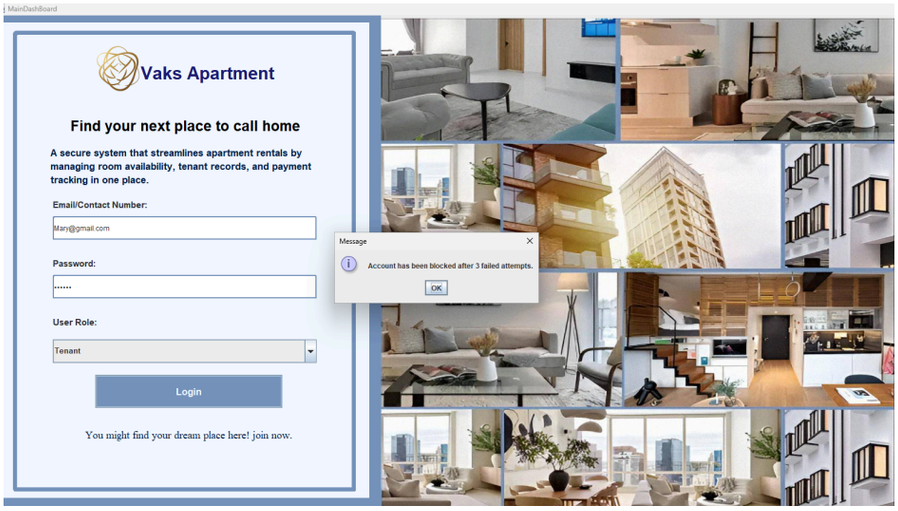


Figure 2

Implements a 3-attempt login lockout policy that automatically blocks accounts after 3 consecutive failed passwords to protect against unauthorized access.

2.2 Tenant Home Dashboard

The central hub for logged-in tenants displaying an organized apartment overview.

Key components include:

- **Personalized Greeting:** Welcomes the logged-in tenant by name.

- **Apartment Overview Cards:** Displays lease details at a glance, including:
    - **Unit Number:** Shows the tenant's assigned apartment unit .
    - **Monthly Balance:** Displays current rent amounts due in Philippine Peso.
    - **Lease End Date:** Highlights the expiration date of the tenant's contract.
- **Quick Action Controls:** Provides convenient navigation buttons allowing tenants to seamlessly jump to **View Account** or **Rent and Payment**.
- **Top Navigation Bar:** Allows quick switching between **Home**, **Payments**, and **Account** screens.

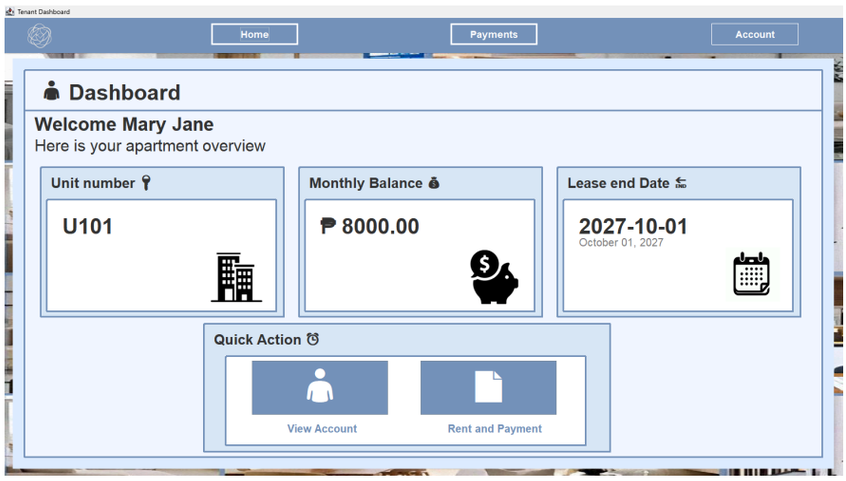


Figure 3

2.3 Rent and Payment

Allows tenants to track their balance due, initiate transactions, and review payment logs.

Key components include:

- **Current Balance Card:** Displays the rent amount (e.g., ₱ 8000.00) alongside the exact due date (e.g., November 01, 2026).
- **Payment Action Controls:** Includes a **Pay Now** button to quickly settle current outstanding balances, and a **Pay Overdue** button for processing overdue balances .
- **Payment History Table:** Logs past and ongoing transactions with columns for Date, payment Method, Amount, payment Status, and Tenant ID.
- **Receipt Viewer:** Features a **View Receipt** button that lets tenants inspect their billing receipts for confirmed transactions.

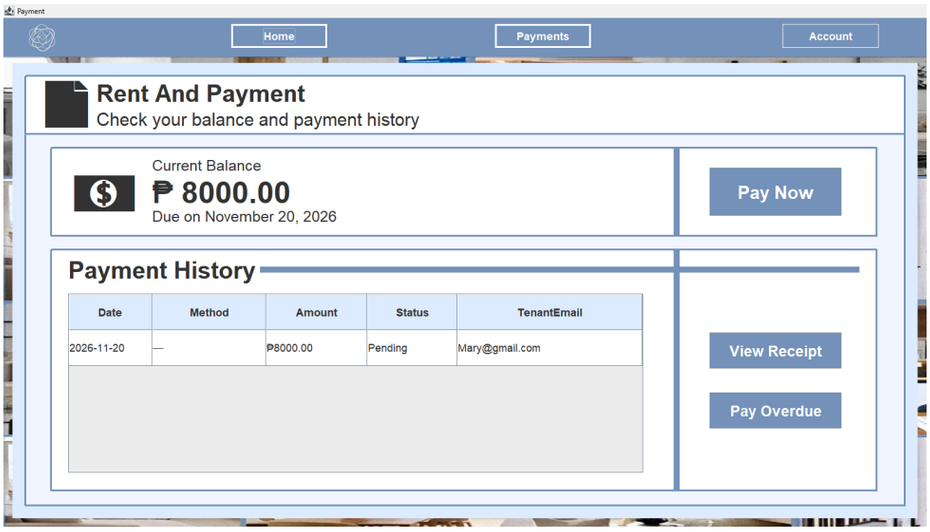


Figure 4

2.4 Account Management (User Profile)

Provides tenants with a clear overview of their personal records, lease terms, and account controls.

Key components include:

- **Personal Information Panel:** Displays tenant identification details including Full Name, Gender, Tenant ID, Email, and Contact Number.
- **Occupancy Details Panel:** Displays active lease details including Apartment Unit, Unit Type, Monthly Rent, Stay Duration, Move-in Date, and Lease End Date.
- **Account Controls:** Features action buttons for tenants to **Edit Profile** details, **Change Password** for account security, or **Log Out** of the session.

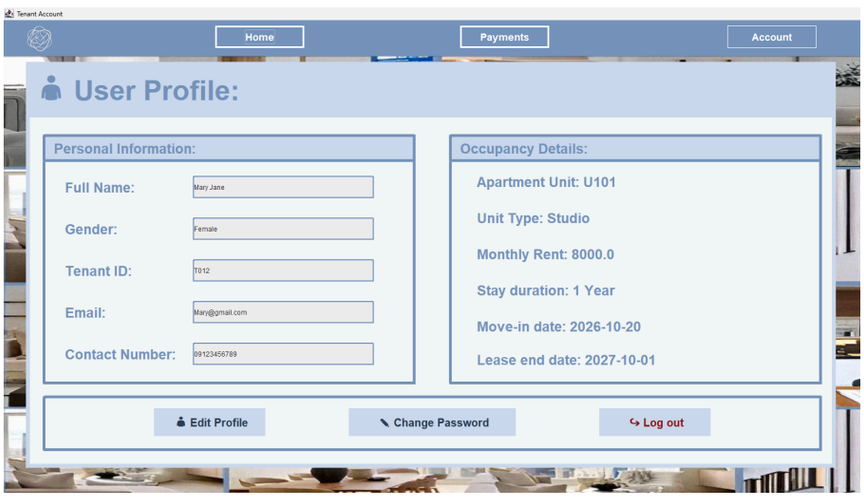


Figure 5

2.5 Administrator Dashboard

Serves as the main control center for system administrators to monitor all property operations, track occupancy, and manage property records.

Key components include:

- **Property Overview:** Summary cards that display real-time overview including Total Tenants, Occupied Units, and Available Units.
- **Recent Transactions Log:** A table displaying recent financial activity with columns for Date, Tenant, Payment Method, and Amount.
- **Quick Action Controls:** Shortcut buttons allowing administrators to execute common tasks instantly:
    - **Add Tenant:** Registers a new tenant into the system.
    - **Add Unit:** Creates a new rental unit record.
    - **Tenant Management:** Opens tenant management and account lists.
    - **Unit Management:** Displays full unit availability and occupancy records.
- **Admin Navigation Bar:** Provides seamless access across major administrative modules including **Home**, **Tenant Management**, **Unit Management**, **Transaction History**, and **Logout**.

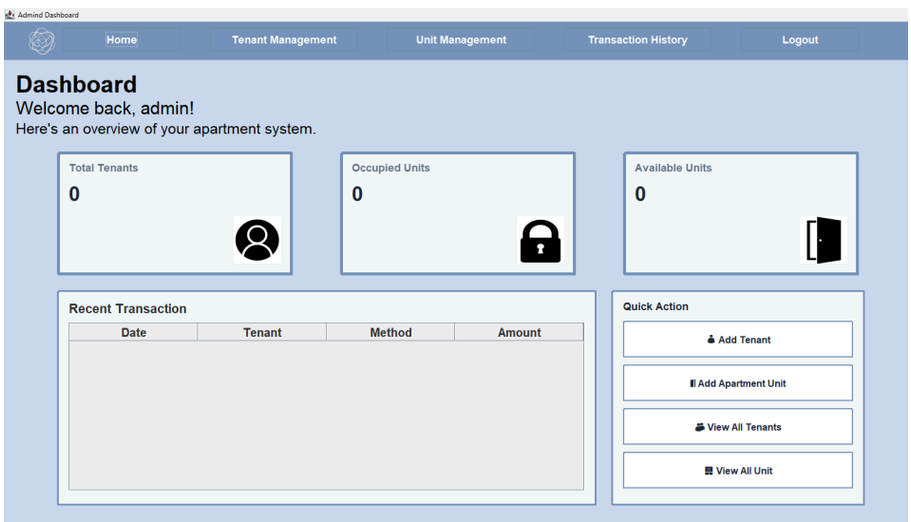


Figure 6

2.6 Interactive Pop-ups and Dialog Windows

- **Add & Edit Tenant Dialog:**
    - Triggered from the Tenant Management dashboard.

- Displays input fields for personal information and assigns active apartment units, move-in dates, and lease duration.

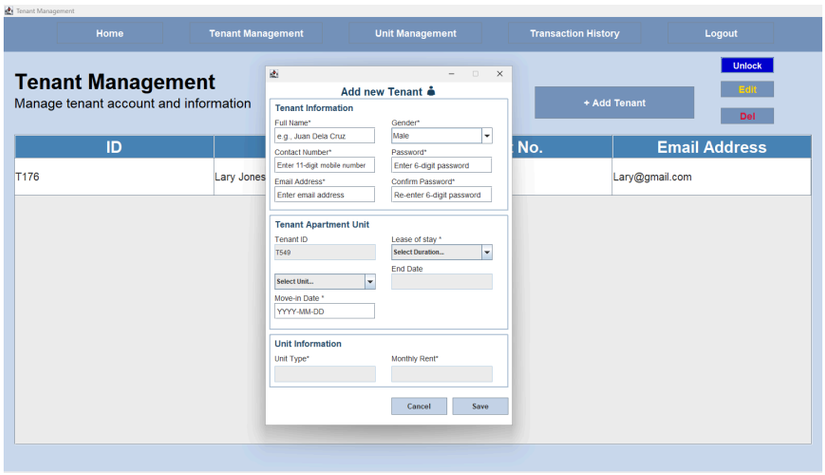


Figure 7

- **Add & Edit Unit Dialog:**
  - Accessible by Administrators on the Unit Management screen.
  - Allows admins to update unit numbers, unit types, floor area, monthly rent amounts, and availability status.

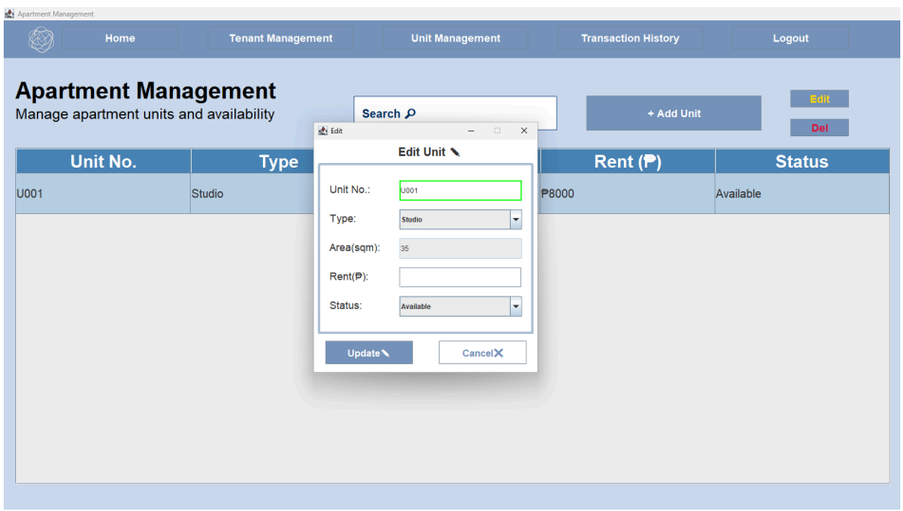


Figure 8

- **Account Security & Credential Pop-ups:**
  - Change Password Dialog: Lets tenants securely update credentials by confirming their current email, old password, and new 6-digit password.
  - Account Lockout Notification: Triggered automatically by the system when a user fails 3 consecutive login attempts, disabling account access until administrative action is taken.

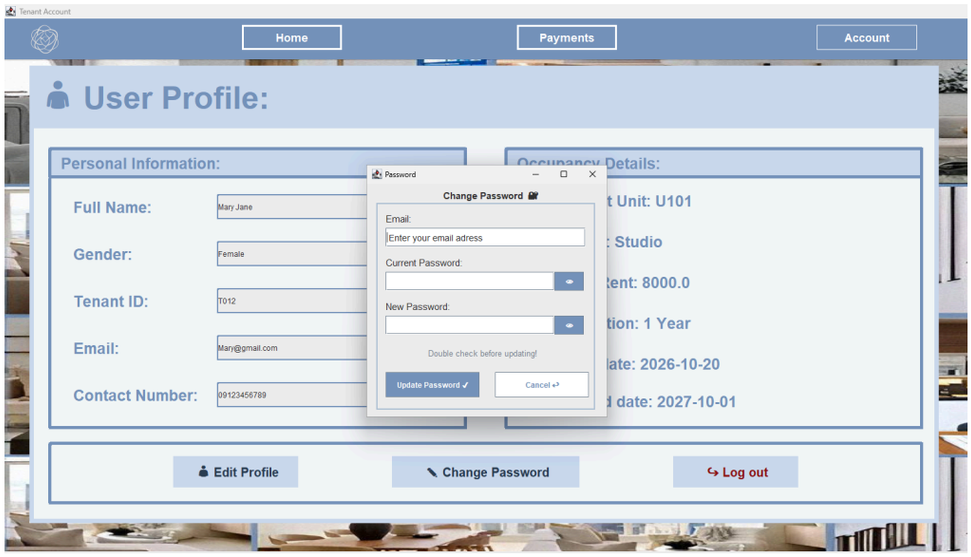


Figure 9

- **System Action Confirmations & Alert Dialogs**:
    - Displays feedback messages for successfully updated records, invalid inputs , or critical delete actions.

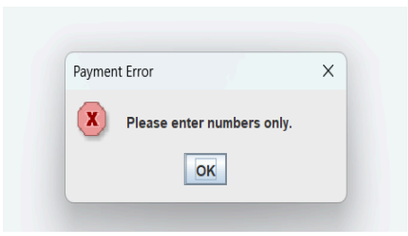


Figure 10.1

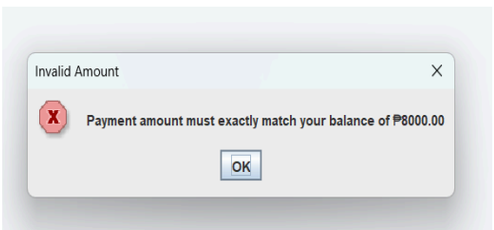


Figure 10.2

3. OBJECT-ORIENTED PROGRAMMING IMPLEMENTATION

3.1 Classes and Objects

The Vaks Apartment Management System breaks things down into smaller, organized pieces called classes. Each piece has its own specific job to make the app easy to run and manage.

Key Classes:

- **Main.java :** Acts as the lightweight execution driver that launches the application by instantiating MainExtended.
- **MainExtended.java:** Acts as the main command hub that manages login access and keeps all parts of the app connected to the active user session and user database.
- **UserDataBP.java:** Serves as the data blueprint class that encapsulates individual tenant and administrator records.

- **DataStore.java:** Manages centralized shared table models for units, tenants, and financial transactions across administrative and tenant views.

Object Creation Sample:

public static UserData userData = **new** UserData();

public static UserDataBP adminAccount = **new** UserDataBP("Admin User", "Admin@gmail.com", "325415", "09123456789", "Admin");

3.2 Encapsulation

Encapsulation is greatly used within **UserDataBP.java** and data models to protect important account fields and variables:

- Attributes such as **passwords, failed login attempt counters,** and **block timestamps** are kept secure and accessed exclusively through public getter and setter methods
- This protects internal data from unauthorized manipulation, ensuring that security policies like the 3-attempt account lockout mechanism execute reliably.

3.3 Inheritance

- All major window screens like **MainExtended**, **AdminHome**, **ApartmentManagement**, **Account**, and **Home** extend Java's built-in **JFrame** class.
- By extending **JFrame,** these panels automatically get built-in window controls, smooth screen drawing, and layout management right out of the box.

3.4 Polymorphism and Event Handling

Polymorphism and dynamic event handling are used across user interactions:

- **Event listeners** uses modern functional interfaces and lambda expressions (e.g., button1.addActionListener(e -> { . })) to handle button clicks and route navigation dynamically at runtime.
- Real-time text validation utilizes **DocumentListener** implementations across input fields in classes **like ChangePassUI, EditTenant, and JTFF,** polymorphically executing text checking rules (insertUpdate, removeUpdate, changedUpdate) whenever user input changes.

4. DATABASE AND DATA MANAGEMENT

4. DATABASE AND DATA MANAGEMENT

The Vaks Apartment Management System stores its data in memory instead of using an external database. User accounts are kept in an ArrayList, and the tenant, unit, and payment records are kept in table models, so no database server is needed.

4.1 Centralized Data Store (DataStore.java)

DataStore.java holds the system's tenant list, apartment units, and transaction logs in one place.

- It creates and keeps shared DefaultTableModel objects (tenantModel, unitModel, and paymentHistoryModel) that feed the JTable components on both the admin and tenant screens.
- Registered users are stored in an ArrayList of UserDataBP objects, managed by UserData.java (see 4.2).
- Because every screen reads from the same models, changes made by the admin, like adding a unit or updating a tenant, show up on all screens right away.

4.2 User and Account Management (UserData.java and UserDataBP.java)

Tenant authentication and account records are controlled through a structured object collection:

- **UserData.java** manages an ArrayList collection of registered users, providing search and query methods (such as findByEmailOrPhone()) to process login credentials and account lookups.
- **UserDataBP.java** acts as the individual record blueprint, storing critical attributes like user roles, contact details, lease terms, and security states

4.3 Data Synchronization and Updates

To maintain data consistency across different panels, the application implements custom synchronization logic:

- When an administrator edits tenant records or payment statuses, helper methods like updateTenantDetails() automatically propagate changes across history logs and transaction tables.
- Input validation rules (enforced via DocumentListener checks) ensure that data formatting such as phone number structures, email formats, and 6-digit password constraints remains accurate before saving.

5. CONCLUSION

The **Vaks Apartment Management System** shows how Object-Oriented Programming and Java Swing can be used to build a desktop app for managing apartments. Admins can handle tenants, units, and transactions, while tenants can check their balances, payments, and account details. The code is split into classes, **DataStore** shares data between screens, and logins are routed by role.

The hardest part was keeping the screens in sync. Fixing one bug often caused a new one in another panel. For example, using static variables kept the previous state, preventing screens from updating properly. We solved this by consolidating all data management into a single centralized class, ensuring that screen data updates seamlessly across all panels.

Security and data quality are ultimately kept in check through features like automatic 3-attempt account lockouts, live input checks, and automatic screen updates. Overall, the system overcomes these obstacles to meet its goals, offering a reliable and user-friendly digital tool for modern rental management.